

OSSP (Operating Systems and Systems Programming) – 25CS2104E

OSSP LAB – PRACTICAL-3

Aim

To develop a C program using fork() to create a parent and child process, display their Process ID (PID), Parent Process ID (PPID), and process states at different stages, and design an experiment using Linux monitoring tools such as ps, top, and /proc to observe process state transitions between Ready, Running, Waiting, and Terminated.

Algorithm

1. Start the program.
2. Display the PID and PPID of the initial parent process.
3. Call fork() to create a child process.
4. Check the return value of fork():
 - If it is negative, display an error.
 - If it is zero, execute the child process.
 - If it is positive, execute the parent process.
5. In the child process, display its PID and PPID and perform some operations with a delay to observe its state.
6. In the parent process, display its PID and PPID and wait for the child using wait().
7. Use ps to observe the PID, PPID, and current process state.
8. Use top to continuously monitor the processes and observe their CPU activity and states.
9. Use /proc/<PID>/status or /proc/<PID>/stat to obtain detailed process-state information.
10. Observe the process during different stages:
 - **Ready:** process is waiting for CPU scheduling.
 - **Running:** process is currently executing.
 - **Waiting:** process is waiting for an event or resource.
 - **Terminated:** process has completed execution.
11. Record the observations from ps, top, and /proc.
12. After the child terminates, the parent continues and terminates.

13. Stop the program.

Steps

Step 1

```
bhavana@Bhavana:~$ nano process_states.c
```

Step 2

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

void display_process_info(const char *stage)
{
    char path[50];
    char state;
    FILE *fp;

    snprintf(path, sizeof(path), "/proc/%d/stat", getpid());

    fp = fopen(path, "r");

    if (fp != NULL)
    {
        fscanf(fp, "%*d %*s %c", &state);
        fclose(fp);
    }
    else
    {
        state = '?';
    }

    printf("\n[%s]\n", stage);
    printf("PID : %d\n", getpid());
    printf("PPID : %d\n", getppid());
    printf("State: %c\n", state);
}

int main()
{
    pid_t pid;

    printf("Parent process started.\n");

    ^G Help      ^O Write Out  ^F Where Is   ^K Cut
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste
```

Step 3

```
bhavana@Bhavana:~$ gcc process_states.c -o process_states
bhavana@Bhavana:~$ ls
command_exec  permission_test.txt  process.c.save  process_states.c  readfile  sample.txt  student.txt  task1.c
command_exec.c  practical2.c        process_states  q1.c              readfile.c  skill1.1    task1
```

Step 4

```
bhavana@Bhavana:~$ ./process_states
Parent process started.

[Parent - Before fork()]
PID : 1067
PPID : 978
State: R

[Parent - After fork()]
PID : 1067
PPID : 978
State: R

Parent is waiting for child...

[Child - After fork()]
PID : 1068
PPID : 1067
State: R

Child is running...

[Child - After sleep()]
PID : 1068
PPID : 1067
State: R

Child process terminating...

[Parent - After wait()]
PID : 1067
PPID : 978
State: R

Parent process terminating...
bhavana@Bhavana:~$ |
```

Step 5

```

Parent process terminating...
bhavana@Bhavana:~$ ps -o pid,ppid,state,cmd
  PID    PPID  S  CMD
   329    326  S  -bash
   555    329  R  ps -o pid,ppid,state,cmd

```

Step 6

```

bhavana@Bhavana:~$ top
top - 06:54:53 up 4 min,  1 user,  load average: 0.04, 0.04, 0.00
Tasks:  25 total,   1 running,  24 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,   0.0 sy,   0.0 ni,100.0 id,   0.0 wa,   0.0 hi,   0.0 si,   0.0 st
MiB Mem :  7784.6 total,  6618.2 free,   533.6 used,   784.3 buff/cache
MiB Swap:  2048.0 total,  2048.0 free,    0.0 used,  7251.0 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
    1 root        20   0   24216   15380  11596 S   0.0   0.2   0:00.73 systemd
    2 root        20   0    3180    2212   2072 S   0.0   0.0   0:00.01 init-systemd(Ub
    8 root        20   0    3180    2040   1940 S   0.0   0.0   0:00.00 init
   48 root        19  -1   42204   16512  15236 S   0.0   0.2   0:00.20 systemd-journal
   80 systemd+    20   0   22416  144448  11956 S   0.0   0.2   0:00.09 systemd-resolve
   90 root        20   0   35380   12356   9252 S   0.0   0.2   0:00.34 systemd-udev
  109 root        20   0    2888    1948   1820 S   0.0   0.0   0:00.05 chronyd-starter
  110 root        20   0    4472    3136   2892 S   0.0   0.0   0:00.01 cron
  111 message+    20   0    8796    5416   4736 S   0.0   0.1   0:00.07 dbus-daemon
  146 root        20   0   44096  29756  14612 S   0.0   0.4   0:00.32 networkd-dispat
  162 root        20   0   18436   9448   8204 S   0.0   0.1   0:00.11 systemd-logind
  165 root        20   0 1792300  15288  12552 S   0.0   0.2   0:00.16 wsl-pro-service
  191 syslog      20   0  220548   5884   4604 S   0.0   0.1   0:00.09 rsyslogd
  213 root        20   0    5492    2712   2528 S   0.0   0.0   0:00.00agetty
  218 _chrony      20   0   23736  11148   7148 S   0.0   0.1   0:00.08 chronyd
  233 root        20   0  123460  32828  18220 S   0.0   0.4   0:00.26 unattended-upgr
  238 _chrony      20   0   12096    2456   1812 S   0.0   0.0   0:00.00 chronyd
  325 root        20   0    3184    1116    980 S   0.0   0.0   0:00.00 SessionLeader
  326 root        20   0    3200    1252   1108 S   0.0   0.0   0:00.07 Relay(329)
  329 bhavana     20   0    6268    5536   3860 S   0.0   0.1   0:00.12 bash
  333 root        20   0    8648    5536   4692 S   0.0   0.1   0:00.01 login
  383 bhavana     20   0   22344  13444  10820 S   0.0   0.2   0:00.16 systemd

```

Step 7

```
bhavana@Bhavana:~$ ./a.out
```

```
Parent process started.
```

```
[Parent - Before fork()]
```

```
PID : 704
```

```
PPID : 329
```

```
State: R
```

```
[Parent - After fork()]
```

```
PID : 704
```

```
PPID : 329
```

```
State: R
```

```
Parent is waiting for child...
```

```
[Child - After fork()]
```

```
PID : 705
```

```
PPID : 704
```

```
State: R
```

```
Child is running...
```

```
[Child - After sleep()]
```

```
PID : 705
```

```
PPID : 704
```

```
State: R
```

```
Child process terminating...
```

```
[Parent - After wait()]
```

```
PID : 704
```

```
PPID : 329
```

```
State: R
```

```
Parent process terminating...
```

```
bhavana@Bhavana:~$
```

Conclusion

The experiment successfully demonstrated the creation and execution of parent and child processes using `fork()`. The PID, PPID, and process states were observed at different stages using `ps`, `top`, and `/proc`. The experiment showed how a process transitions through Ready, Running, Waiting, and Terminated states during its lifetime, providing a practical understanding of Linux process management and scheduling.